

And finally, looking at my repo on github, I understand that there is some fintech or financial-based projects. Do you think in regards to the type analysis that they're for fintech or finance, can any of them be addressed? Now this is a completely separate.

Réfléchi pendant 10s

Yes. I looked directly through your GitHub repositories, and **you actually have a surprisingly strong finance/fintech cluster already**. I would keep this completely separate from the library-showcase project we were discussing.

The strongest ones are:

Repository	Finance relevance	What it demonstrates
Credit-Card-Transactions-Fraud-Detection-Project	★★★★★	Fraud detection, transaction analytics, risk modelling, SQL/PostgreSQL
Stock-Financial-Dashboard	★★★★★	Market analytics, technical indicators, financial visualisation
SUI-Crypto-analysis	★★★★★	Financial NLP, sentiment analysis, price prediction, XGBoost
Binance-Crypto-Prediction-and-Analysis	★★★★★	Real-time financial APIs, time series, LSTM forecasting
Forecasting-Ethereum-with-an-LSTM	★★★★★	Financial time-series forecasting / deep learning
xrp_sentiment_analysis	★★★★★	Market sentiment + NLP

Repository	Finance relevance	What it demonstrates
UK-financial-spending	★★★★	Personal/consumer financial analytics
Project-UK-Student-Debt-and-Finance-Dashboard	★★★★	Consumer finance / debt analytics
House-Property-Sales-Analysis	★★★	Property/real-estate finance
fcc-Mean-Variance-Standard-Deviation-Calculator	★★★	Statistical/portfolio-analysis foundations

The really interesting part

Your **fraud project** is probably the most naturally "fintech" one.

The dataset is explicitly simulated credit-card transaction data containing customers, merchants, transaction amounts, locations, transaction times and a fraud indicator. Your README already identifies fraud detection, behavioural modelling, risk scoring and time-series analysis as use cases.

That can be framed much more professionally as:

Financial Crime & Transaction Risk Analytics

rather than simply "I trained a fraud model."

You could potentially expand it into:

Transactions → Data Quality → Behavioural Features → Fraud/Risk Model → Explainability → Risk Dashboard → Analyst Investigation

That starts looking like an actual fintech analytics workflow.

Your Stock Financial Dashboard is also very usable

This one is currently much more of a visualisation application: two stocks, candlesticks, date selection, 30/100-day SMA and linear regression trends.

But the underlying analytical problem is absolutely financial.

You could turn it into something considerably more sophisticated:

Market Data

- returns
- volatility
- drawdown
- moving averages
- momentum
- correlation
- risk metrics
- portfolio comparison
- interactive dashboard

That would move it from "**stock charting app**" to "**Financial Markets Analytics Platform.**"

And your crypto projects are more valuable than they might initially appear

Your SUI project combines financial news scraping, NLP sentiment analysis, historical prices and XGBoost prediction.

Your Binance project goes further into financial data engineering: it connects to the Binance API, retrieves BTC/ETH data, preprocesses it, visualises it and applies an LSTM to financial time-series data.

So you've effectively already got pieces of:

- **Market data engineering**
- **Financial time-series analysis**
- **Quantitative analysis**
- **NLP / alternative data**
- **Forecasting**
- **Risk analytics**
- **Fraud detection**
- **Financial dashboards**

That's actually a pretty good foundation for a **finance/fintech analytics portfolio**.

I would *not* try to force all of them into one project

This is the important distinction.

Your previous idea was:

"Can I build one huge demonstration that showcases all these Python/data-analysis libraries?"

That's a good reason to combine things.

But for your finance work, I'd actually do the opposite:

Keep the finance projects separate, but make each one substantially better.

You could end up with something like:

FINANCE / FINTECH PORTFOLIO

01 — Financial Crime Analytics

Credit-card transactions

→ Fraud detection

→ Risk scoring

→ Explainability

→ PostgreSQL

→ Power BI / Streamlit

02 — Financial Markets Analytics

Stock dashboard

→ Returns

→ Volatility

→ Drawdowns

→ Technical indicators

→ Portfolio analysis

03 — Crypto Market Intelligence

SUI / XRP

→ Financial news

→ NLP sentiment

→ Market movements

→ XGBoost

→ Time-series analysis

04 — Financial Forecasting

BTC / ETH

→ API ingestion

→ Time-series feature engineering

→ LSTM

→ Forecast evaluation

05 — Consumer Finance Analytics

UK spending / student finance

→ Spending behaviour

→ Debt

→ affordability

→ demographic/economic analysis

That would give you **breadth across financial analytics**, rather than five projects that all basically say "I predicted a cryptocurrency."

And importantly, this isn't me suggesting you invent a finance background you don't have. **The repositories genuinely already contain those analytical themes.** The opportunity is mostly to modernise the analysis, improve the engineering, introduce stronger business questions, and make the README/reporting tell the story properly.

If you're thinking specifically about **fintech/financial-services jobs**, I'd rank the projects I'd invest further effort into as:

1. Credit-card fraud → 2. Stock/market analytics → 3. SUI/XRP financial NLP → 4. Binance/ETH forecasting → 5. UK financial/student-debt analytics.

That would complement your existing **customer intelligence, churn, LTV and data-platform work** extremely well: you'd have both **customer/business analytics** and **financial/risk/market analytics** represented.

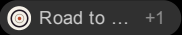
Um, what I'm gonna do, I'm thinking I'm gonna refactor the SUI project and pretty much make it more robust. We might take a look at the top AI companies at the moment. Or I can take a look at the top... actually, I think we can do—we can do more. We can actually make it much more comprehensive. Comprehensive. So, the top AI companies, that being OpenAI, Anthropic, XAI, and Alphabet. Those are the top AI companies. Then we have the Chinese models, so that means DeepSeek, Qwen, Mushort, Kimi, NGLM. I think that's one thing. That's another thing. And then we look at AI stocks. There are AI stocks that we're looking at. For example, we can group them in particular ways. So we can group, for example, Sandisk, ASML, Nvidia, Micron Technologies, Apple, Microsoft, AMD, Taiwanese semiconductor. And then we also have... I'm trying to think what else I can have.

Show less

Yes — **this is a much better direction for the SUI project.** Rather than keeping it as "*SUI sentiment* → *SUI price prediction*", you can turn it into something closer to an **AI Industry**

& AI Equity Intelligence platform.

And I think your instinct to split the companies into **AI model providers** and **AI infrastructure/equities** is exactly right.

One correction, though: I wouldn't frame the first group simply as "the top AI companies." The landscape is broader and changes quickly. Current 2026 trackers include OpenAI, Anthropic, Google, xAI, Meta, Mistral, DeepSeek, Alibaba/Qwen, Z.ai/GLM, Moonshot/Kimi and others. 

I'd structure the refactored project like this

1. Frontier AI model companies

US / Western

- OpenAI
- Anthropic
- Google DeepMind / Alphabet
- xAI
- Meta
- Microsoft
- Mistral

Chinese

- DeepSeek
- Alibaba / Qwen
- Moonshot AI / Kimi
- Zhipu AI / GLM
- Baidu / ERNIE
- MiniMax
- Tencent

That's actually more interesting analytically than just OpenAI vs Anthropic. You can compare **model releases, benchmarks, sentiment, pricing, adoption and company financial performance**.

For example, current LLM market data shows meaningful differences in revenue share between Anthropic, OpenAI, Google, Microsoft and Chinese providers, so there's an actual quantitative business question here rather than just a model leaderboard. 

2. AI Infrastructure & Semiconductor Equities

This is where your stock list becomes really interesting.

I wouldn't just have:

NVDA, AMD, ASML, Micron...

I'd build an **AI supply chain taxonomy**.

Compute

- NVIDIA
- AMD
- Broadcom
- Intel

Foundry / manufacturing

- TSMC
- ASML
- Applied Materials
- Lam Research

Memory / storage

- Micron
- SK Hynix
- Samsung
- SanDisk
- Seagate

Networking

- Broadcom
- Arista Networks
- Cisco
- Marvell

AI/cloud infrastructure

- Microsoft
- Alphabet
- Amazon
- Oracle
- Meta
- CoreWeave
- Nebius

Consumer / AI platforms

- Apple
- Microsoft
- Alphabet
- Amazon
- Meta

This gives you a **value-chain model of AI** rather than a random basket of "AI stocks."

And there's a legitimate analytical argument behind this categorisation: AI spending isn't only benefiting GPU manufacturers. Memory, networking, custom silicon and semiconductor manufacturing are becoming increasingly important parts of the infrastructure stack.  The Motley F... +1

Broadcom is a particularly good addition because its AI exposure isn't simply "another GPU company": its custom accelerators and networking business are directly tied to hyperscaler AI infrastructure.  Reut...

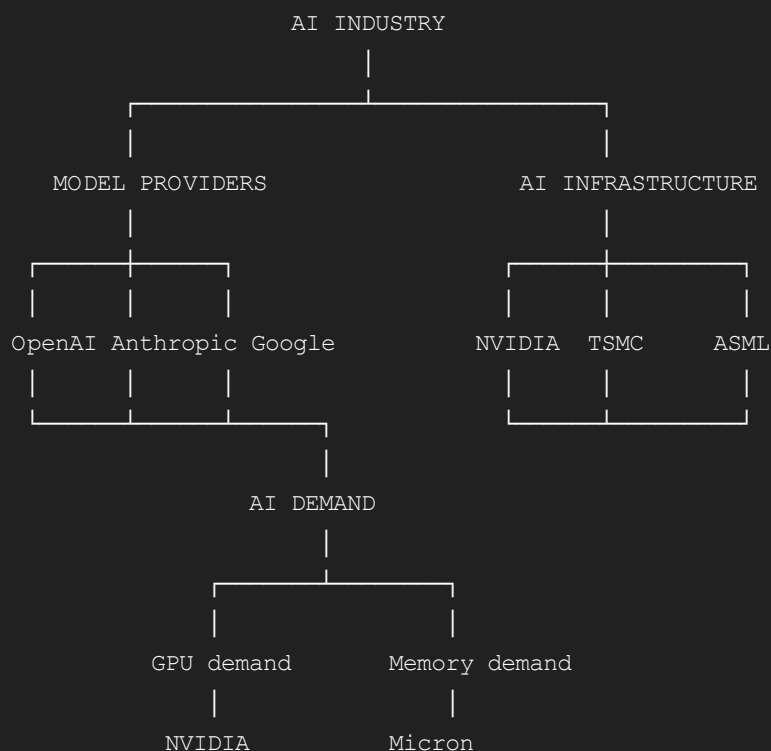
3. Then I'd add an AI ecosystem layer

This is where I think the project could become genuinely impressive.

You could model relationships like:

AI Company → Model → Infrastructure → Suppliers → Stock → Market Reaction

For example:



AMD

SK Hynix

Broadcom

SanDisk

Now you've got something that is **much more than a cryptocurrency project**.

4. Keep the original SUI functionality

And this is important: **don't throw away what already exists**.

Your current SUI repository already has the architecture for:

Financial news → scraping → NLP → sentiment → market data → XGBoost → price prediction.

Instead, generalise it.

Rename the conceptual pipeline from:

SUI Crypto Sentiment Analysis

to something like:

AI Market Intelligence & Financial Analytics

Then SUI becomes **one asset in the system**, rather than the entire subject.

5. The datasets become much richer

You could have:

`companies`

```
company
ticker
country
sector
category
market_cap
```

`models`

```
model
company
release_date
model_type
```

```
context_window
open_weights
benchmark_scores
```

news

```
timestamp
company
headline
source
url
sentiment
sentiment_score
```

market_data

```
timestamp
ticker
open
high
low
close
volume
```

model_benchmarks

```
model
benchmark
score
date
```

ai_relationships

```
company
supplier
relationship_type
dependency
```

And then you can create derived metrics.

6. This gives you some genuinely good analytical questions

Instead of:

"Can sentiment predict SU's price?"

you can ask:

Does positive AI news sentiment translate into abnormal stock returns?

Which AI companies generate the most positive/negative media sentiment?

Does model-release sentiment affect the stock price of associated companies?

Do semiconductor stocks move together during major AI announcements?

Which segment of the AI supply chain has the highest volatility?

Does AI infrastructure sentiment lead semiconductor returns?

Are NVIDIA, AMD, TSMC, ASML and Micron becoming increasingly correlated?

Does Chinese AI news produce different market reactions from US AI news?

Does a major model release generate measurable abnormal returns?

That's **proper financial analytics**.

And there's one feature I'd REALLY add

An AI Investment Theme Index

Create your own synthetic index:

AI Infrastructure Index

For example:

- NVIDIA — 20%
- TSMC — 15%
- Broadcom — 15%
- ASML — 10%
- AMD — 10%
- Micron — 10%
- SK Hynix — 5%
- SanDisk — 5%
- Applied Materials — 5%
- Arista — 5%

Then another:

AI Platform Index

and perhaps:

AI Consumer Index

Then compare them against:

- S&P 500
- Nasdaq-100
- SOX / semiconductor benchmark
- Bitcoin
- Ethereum

You can analyse:

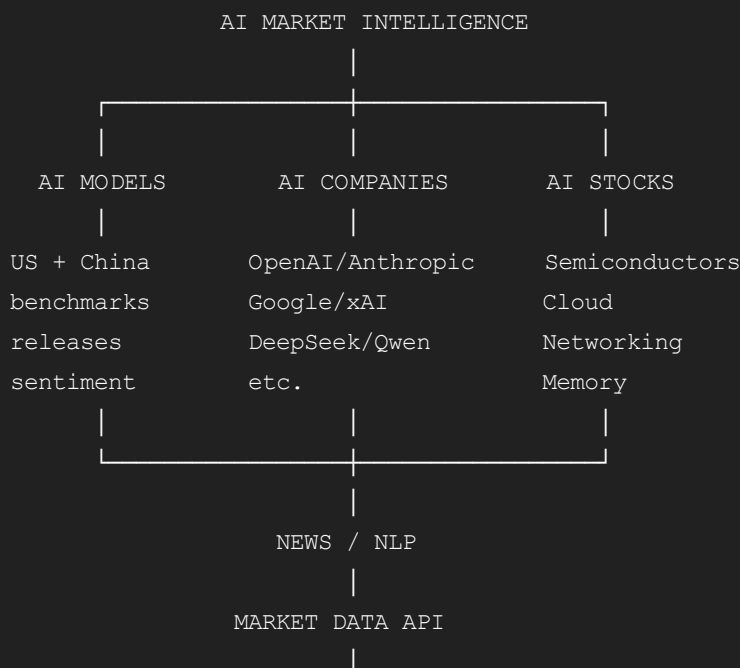
returns → **volatility** → **Sharpe** → **maximum drawdown** → **beta** → **correlation** → **rolling correlation** → **sentiment** → **event reactions**.

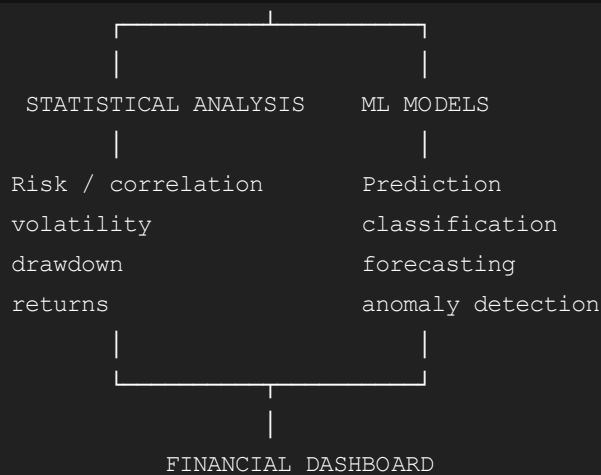
That transforms the project into something that looks much closer to a **quantitative financial research project**.

And it's particularly timely: AI-related semiconductor stocks have been moving as a group recently, with NVIDIA, Micron, SanDisk, TSMC and others responding to changing expectations around AI infrastructure spending.  Reut... +1

So I'd ultimately turn your repo into:

AI Market Intelligence Platform





That is substantially more impressive than the original SUI project.

And crucially, you don't have to abandon the original project — you're essentially **turning SUI into the prototype for a much larger financial intelligence system.**

Now, in my previous model, I used XGBoost, and this is a classification model. In order to see which model performs the best, I think we should compare them. So we should do a logistical regression, random forest classifier, XGBoost, and if there's any other models that I should be aware of, they should be included as the fourth model to test. If not, three would be fine.

Yes — I would absolutely make this a **model comparison exercise**, rather than simply carrying XGBoost forward because it worked previously.

For the classification component of the refactored project, I'd use **four models**:

Model	Why include it	Role
Logistic Regression	Simple, interpretable baseline	Linear baseline
Random Forest	Captures nonlinear relationships without boosting	Bagging/tree baseline
XGBoost	Strong gradient-boosting model for tabular data	Primary high-performance model

Model	Why include it	Role
HistGradientBoosting Classifier	Independent gradient-boosting implementation	Additional boosting benchmark

Scikit-learn specifically describes gradient-boosted trees as strong models for classification on tabular data, while `HistGradientBoostingClassifier` is designed to be considerably faster on larger datasets.  Scikit-learn +1

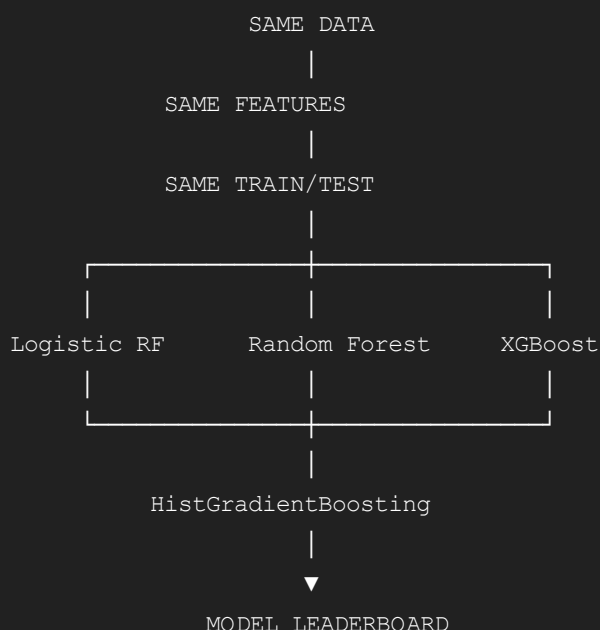
Why I prefer HistGradientBoosting for the fourth model

I wouldn't add a neural network just for the sake of having a fourth model. Your data will predominantly be structured/tabular:

```
sentiment
price returns
volume
volatility
technical indicators
market cap
company category
AI model/release events
etc.
```

So a fourth tree-based model gives you a much more meaningful comparison.

The experiment becomes:



And importantly, **don't just compare accuracy.**

For your financial classification problem I'd report:

- Accuracy
- Precision
- Recall
- F1
- ROC-AUC
- PR-AUC
- Confusion matrix
- Training time
- Prediction time
- Calibration / probability quality
- Feature importance / interpretability

If the positive class is something like:

"Will this asset experience a positive abnormal return over the next N days?"

then **ROC-AUC** and **PR-AUC** become particularly useful, while recall/precision tell you what you're actually sacrificing when identifying positive opportunities.

One thing I'd change from the original SUI project

I'd also make the classification target much more financially meaningful.

Instead of:

Predict whether tomorrow's price goes up/down.

I'd consider something like:

Predict whether an asset will generate a positive return exceeding a defined threshold over the next 1/5/10 trading days.

For example:

```
Target = 1
if future_5d_return > 2%

Target = 0
otherwise
```

Now you're not merely predicting **direction**. You're attempting to identify **meaningful market movements**.

That also gives you a much better story for the portfolio:

"I evaluated four classification algorithms to determine whether financial news sentiment, market characteristics and AI-industry signals could identify assets likely to experience significant short-term returns."

That's substantially stronger than:

"I used XGBoost to predict stock prices."

And because you're comparing fundamentally different approaches—linear logistic regression, bagging via Random Forest, and two gradient-boosting approaches—you'll be able to discuss **why one model wins**, rather than simply reporting that XGBoost had the highest score. Random forests reduce variance through independently constructed trees, while boosting sequentially builds trees to improve previous errors.  Scikit-learn +1

So I'd lock in four:

Logistic Regression → Random Forest → XGBoost → HistGradientBoosting.

That is enough. I wouldn't go beyond four for the main experiment; otherwise the comparison starts becoming a model zoo rather than a well-designed analysis.

ChatGPT can make mistakes. Check important info.